



Minería de Datos

Flash K-Means

Anna Martina Visone

Nicolás Martínez López

Miller Estiven Barrera Gonzalez

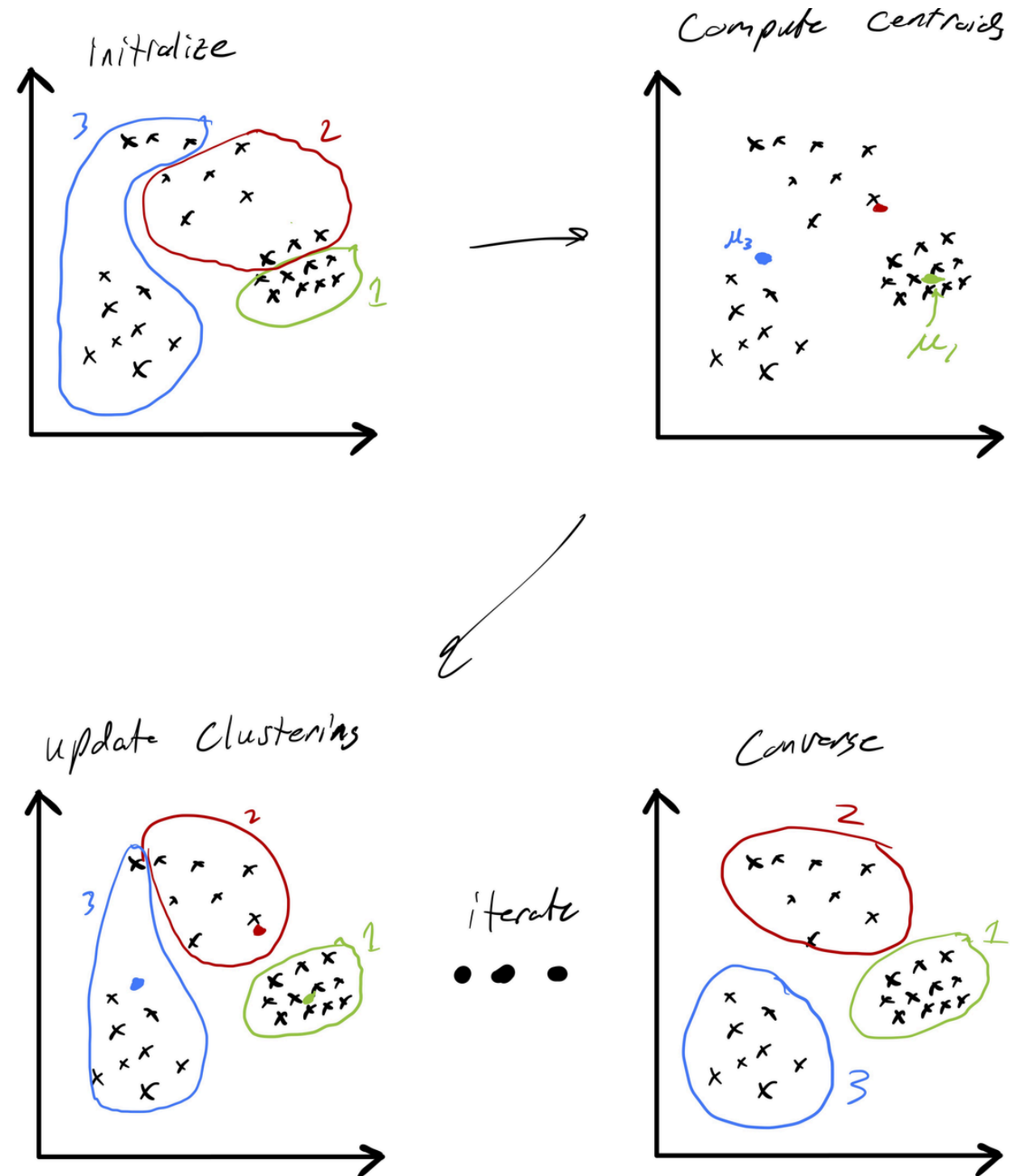
Sergio Andres Castro Vargas

Grupo 5

Universidad Nacional de Colombia
Sede Bogotá



K-MEANS



El algoritmo K-means:

1. Inicialización: Eliges: puntos al azar como centros iniciales (centroides).
2. Asignación: Cada dato del conjunto se une al centroide que tenga más cerca.
3. Actualización: Calculas el promedio de los puntos de cada grupo y mueves el centroide a esa nueva posición.

El proceso se repite hasta que los centros dejan de moverse o los puntos ya no cambian de grupo.

CAMBIO DE PARADIGMA

Antes:

- Minería de datos y visión computacional clásica: Se utilizaba para tareas generales de clasificación y análisis de observaciones
- Organización de conjuntos de datos: Servía para estructurar grandes volúmenes de información antes de otros procesos

Ahora:

- Modelos de Lenguaje (LLMs): Es fundamental para el escalado del contexto, permitiendo el enrutamiento dinámico de tokens para la atención dispersa (sparse attention) y la compresión del caché KV mediante la fusión de estados semánticos
- Modelos Generativos de Video: Se invoca repetidamente durante los pases hacia adelante (forward passes) en Transformers de Difusión para realizar permutaciones de tokens conscientes del contenido semántico

ANATOMIA DEL NUEVO TRABAJO

De CPUs a GPUs: La ejecución ha migrado masivamente hacia los aceleradores de hardware para aprovechar su potencia de cómputo paralelo

De Offline a Online: Debido a que ahora se llama al algoritmo en medio de la generación de una respuesta de IA, la baja latencia por invocación es ahora un requisito estrictamente esencial

De muestras pequeñas a lotes masivos: El cómputo ahora se realiza sobre secuencias mucho más largas y lotes (batches) más grandes, lo que hace que el rendimiento y la escalabilidad sean críticos

FALLAS EN K-MEANS

El Cuello de Botella de IO en la Asignación

Materialización Ineficiente: El sistema calcula primero todas las distancias entre N puntos y K centroides, creando y escribiendo una matriz de distancias masiva ($N \times K$) en la Memoria de Alto Ancho de Banda (HBM)

El impacto real: En una configuración representativa ($N=65536, K=1024$), el cómputo matemático de la distancia toma solo 2.6 milisegundos, pero el tiempo dominado por mover esa matriz en la memoria es de 23 milisegundos. Esto significa que el sistema pasa casi 10 veces más tiempo moviendo datos que procesándolos

FALLAS EN K-MEANS

Restricciones a Nivel de Sistema

Límites de la VRAM y PCIe: Cuando el lote de datos es demasiado grande para residir en la memoria de la GPU (VRAM), el sistema debe procesar por partes, lo que introduce una sobrecarga de comunicación masiva entre el CPU y la GPU a través del bus PCIe

Cargas de Trabajo Dinámicas: La IA moderna utiliza datos con formas que cambian rápidamente (número variable de puntos o clusters). Esto dispara los costos de compilación y ajuste de configuración, haciendo que el tiempo previo a la primera ejecución sea muy alto y afectando la usabilidad práctica en escenarios de tiempo real

FLASH ASSIGN:
ELIMINANDO EL MURO
DE MEMORIA EN K-
MEANS

Problema

**GUARDAR TODA LA MATRIZ DE
DISTANCIA EN HBM Y LUEGO VOLVERLA A
LEER (CARO)**

HBM → compute → guardar D → HBM → leer D → argmin

**El cálculo de distancias puede tomar 2.6 ms,
¡pero mover los datos de la matriz D toma 23
ms!**

Solución

**En lugar de realizar dos pasos separados
(Distancia → Memoria → Argmin), FlashAssign
los fusiona en un solo procedimiento de flujo
continuo (BARATO)**

HBM → compute + argmin en registers/shared
→ guardar solo resultado final

**La matriz de distancias completa nunca
se escribe en la memoria global (HBM)**

FLASH ASSIGN: ASIGNACIÓN SIN MATERIALIZACIÓN MEDIANTE ARGMIN EN LÍNEA

Online argmin: Este concepto se refiere a encontrar el **minimo sin almacenar todos los valores primeros**.

Tiling: Este concepto consiste en dividir los datos (puntos y centroides) en bloques más pequeños para que puedan procesarse eficientemente dentro de la memoria rápida del chip (on-chip) de la GPU.

Precarga asíncrona: Es traer el siguiente bloque de datos desde memoria mientras el bloque actual se está computando, de modo que se solapan carga y cálculo, evitando tiempos de espera y mejorando el rendimiento.

¿CÓMO FUNCIONA?

Online argmin: Para cada punto de datos, se mantienen solo dos estados en los registros: la distancia mínima actual (m_i) y el índice del centroide (a_i). A medida que los centroides pasan por el chip, el estado se actualiza instantáneamente

En el paper...

Para cada punto X_i :

1. Tomas un bloque de centroides $C(t)$
2. Calculas distancias con ese bloque
3. Encuentras el mínimo local
4. Actualizas el mínimo global
5. Pasas al siguiente bloque

Ejemplo

$K = 10,000$ centroides

Para cada punto X_i :

Inicializar $m = +\infty$, $a = -1$

Para $t = 1$ hasta K/BK :

Tomar bloque de centroides $C(t)$

Calcular distancias con $C(t)$

Encontrar mínimo local

Actualizar (m , a)

Guardar a como asignación final

SEGUIR OPTIMIZANDO...

Tiling: Este concepto consiste en dividir los datos (puntos y centroides) en bloques más pequeños para que puedan procesarse eficientemente dentro de la memoria rápida del chip (on-chip) de la GPU.

En el paper...

Para cada bloque de puntos X_{tile} **en paralelo:**

1. Tomas un bloque de centroides $C(t)$
2. Cargas $C(t)$ en memoria rápida (SRAM / registros)
3. Calculas distancias entre X_{tile} y $C(t)$
4. Encuentras el mínimo local
5. Actualizas el mínimo global (m,a)
6. Pasas al siguiente bloque de centroides

Ideas claves

- **Paralelismo de puntos:** Los hilos de la GPU procesan múltiples "bloques de puntos" (X_{tile}) de tamaño BN de forma paralela
- **Escaneo de centroides:** Dentro de cada bloque de hilos, se realiza un bucle sobre los centroides, procesándolos también por bloques (BK). Esto permite que el cálculo de distancias se mantenga dentro del chip (on-chip) sin salir a la memoria global

DOBLE BUFFER

Precarga asíncrona: Mientras los núcleos de cálculo de la GPU están usando el centroide actual en el **Búfer A** para calcular distancias, una unidad de memoria independiente ya está cargando el siguiente bloque de centroides en el **Búfer B**

En el paper...

Para cada bloque de puntos Xtile:

1. **Cargar bloque inicial de centroides $C(1)$**
2. Para cada bloque $C(t)$:
 - Calcular distancias con $C(t)$
 - **Mientras tanto: cargar $C(t+1)$ desde HBM**
 - Continuar sin esperar

Idea clave

Sin latencia: Al terminar el cálculo, la GPU simplemente "intercambia" (swap) los búferes y continúa trabajando sin haber tenido que detenerse a esperar la carga de datos

UNIENDO TODO

Algorithm 2: FlashAssign (materialization-free assignment)

Input: Data $X \in \mathbb{R}^{N \times d}$, centroids $C \in \mathbb{R}^{K \times d}$, point tile size B_N , centroid tile size B_K

Output: Assignments $a \in \{1, \dots, K\}^N$

Precompute norms $\|x_i\|_2^2$;

foreach *point tile* X_{tile} *of size* B_N *in parallel* **do**

 Initialize on-chip running states: $m \leftarrow +\infty$, $a \leftarrow -1$;

 Prefetch the first centroid tile $C_{\text{tile}}^{(0)}$ from HBM into on-chip buffer;

for $t \leftarrow 0$ **to** $\lceil K/B_K \rceil - 1$ **do**

if $t + 1 < \lceil K/B_K \rceil$ **then**

 Prefetch $C_{\text{tile}}^{(t+1)}$ into the alternate buffer;

 Compute local distances between X_{tile} and $C_{\text{tile}}^{(t)}$ on chip;

 Compute tile-local minima and indices $(\tilde{m}, \tilde{a})$ for each point in X_{tile} ;

 Update running states using online argmin;

$m \leftarrow \min(m, \tilde{m})$, and update a with the corresponding index;

 Swap buffers;

 Write final assignments a for X_{tile} to HBM;

MEJORA EN LA COMPLEJIDAD DE I/O

Antes

el algoritmo calcula una matriz de distancias intermedia $D \in \mathbb{R}^{N \times K}$ que guarda y lee en la HBM

Después

la matriz $N \times K$ nunca se escribe en la memoria.


$$2 * O(NK) \rightarrow O(Nd + kd)$$

¿Y SI LA ENTRADA NO CABE ON CHIP?

Problema

¿Qué pasa cuando los datos (ej. 1,000 millones de puntos) no caben en la memoria de la GPU (VRAM)?

Solución

Se debe gestionar el viaje de los datos desde la **CPU (host)** hacia la **GPU(device)**

- Divide el dataset masivo en fragmentos (chunks).
- Usa **CUDA streams** para coordinar transferencias asíncronas.
- Solapamiento Total: Mientras la GPU está calculando el fragmento actual, el bus PCIe ya está cargando el siguiente en paralelo (doble búfer de sistema)

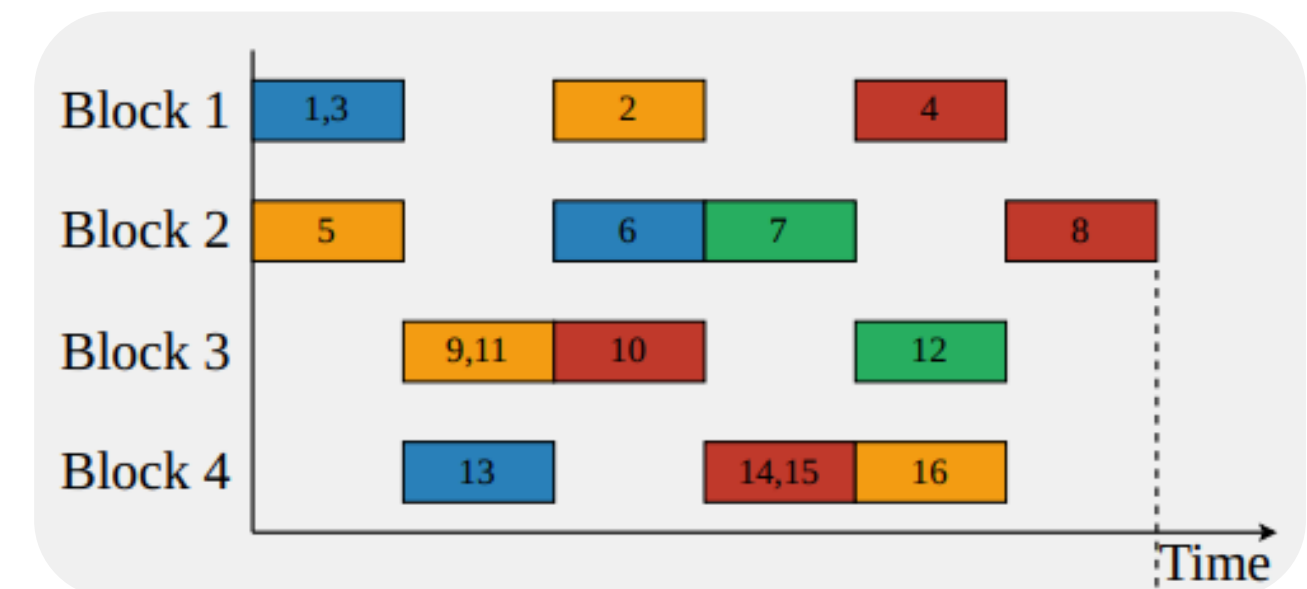
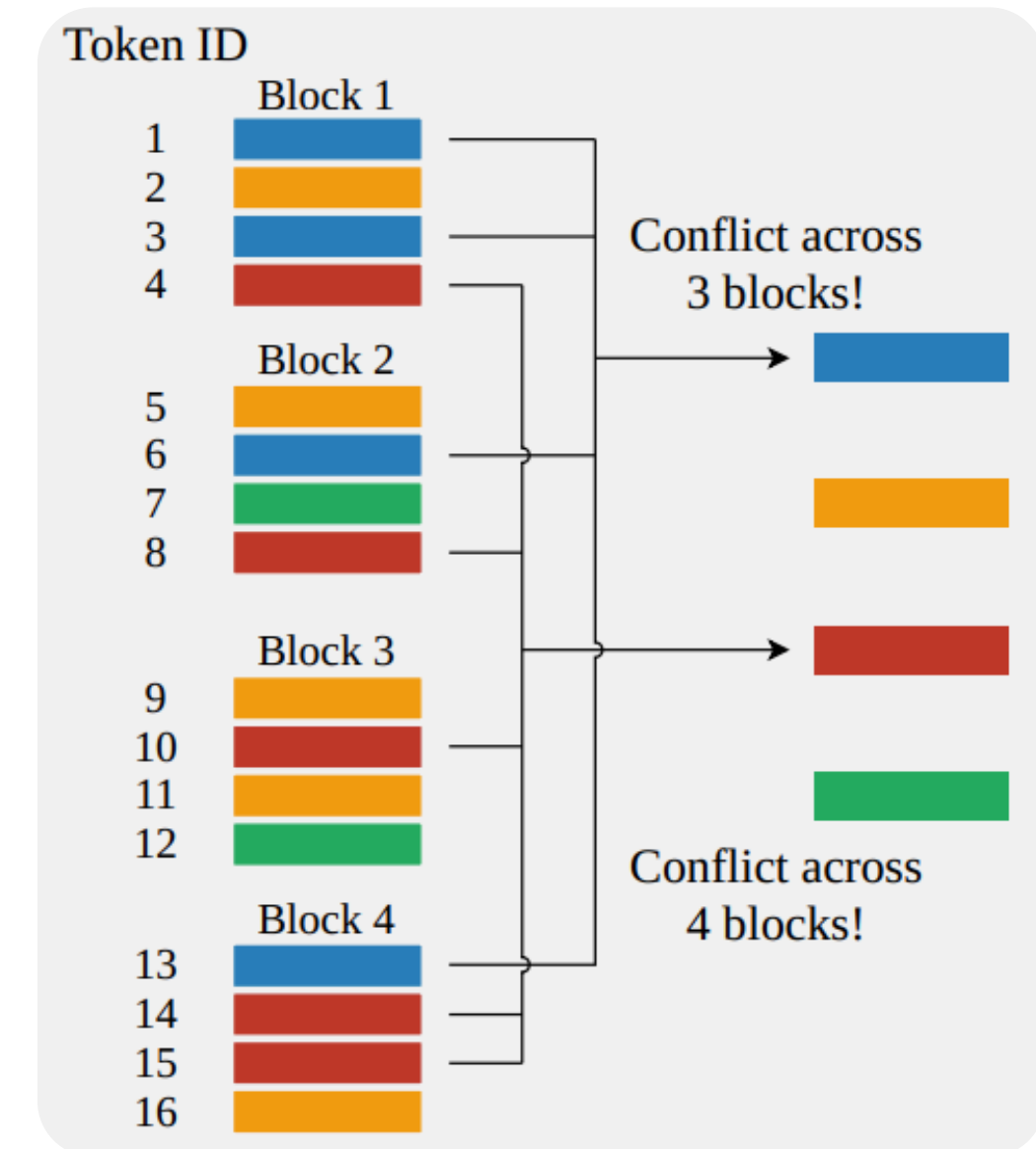
FLASH-K-MEANS:

SORT-INVERSE UPDATE

La Solución a las Peleas Entre Hilos Que Afectan el Desempeño Original del Algoritmo K-Means

PROBLEMA

- En k-means estándar, cada hilo procesa un punto de datos
- Cada hilo calcula a qué clúster pertenece y actualiza ese centroide
- Esto se implementa como un scatter update
- Problema:
 - Múltiples hilos escriben simultáneamente en la misma posición de memoria
- Consecuencia:
 - Conflictos de escritura (race conditions)
 - La GPU Hace uso de Operaciones Atómicas
 - Lentas pero Seguras
 - Serializan el acceso a la memoria
 - Se desperdicia/pierde el paralelismo



Sort-Inverse Update

- En lugar de escribir directamente → reorganizar los datos primero
- Cambio de paradigma:
 - De scatter ← escrituras desordenadas
 - A gather (organizado) ← Agregaciones ordenadas
- Dos Pasos Principales:

**1. Ordenar
Asignaciones**

(Sort)

**2. Agregar por
Segmentos**

(Reduce)

Sort-Inverse Update

Algorithm 3: Sort-Inverse Update (low-atomic centroid update)

Input: Points $X \in \mathbb{R}^{N \times d}$ (original order), assignments $a \in \{1, \dots, K\}^N$, chunk size B_N

Output: Centroid sums $s \in \mathbb{R}^{K \times d}$, counts $n \in \mathbb{R}^K$, updated centroids C

Sort by cluster id: compute $\text{sorted_idx} \leftarrow \text{argsort}(a)$;

Construct sorted cluster ids $a^{\text{sorted}}[j] \leftarrow a[\text{sorted_idx}[j]]$;

Initialize $s \leftarrow 0$, $n \leftarrow 0$;

for $l \leftarrow 0$ **to** $N - 1$ *step* B_N **do**

$r \leftarrow \min(l + B_N, N)$;

 Load $a^{\text{sorted}}[l : r]$ and $\text{sorted_idx}[l : r]$;

 Identify contiguous segments of identical cluster ids in $a^{\text{sorted}}[l : r]$;

for *each segment* (u, v, k) *in* $a^{\text{sorted}}[l : r]$ **do**

 Gather token features from original order using indices $\text{sorted_idx}[u : v]$;

 Accumulate local partial sum Δs_k and local partial count Δn_k on chip;

Atomic merge (once per segment): $\text{atomic_add}(s_k, \Delta s_k)$ and $\text{atomic_add}(n_k, \Delta n_k)$;

for $k \leftarrow 1$ **to** K **do**

$c_k \leftarrow s_k / n_k$;

Sort-Inverse Update

Entrada

- **X** es la matriz de los datos con **N** datos de dimensión **d** cada uno.
- **a** $\in \{1, \dots, K\}^N$
 - **a**: Vector de asignaciones
 - **a_i = k** $\rightarrow$ El punto **x_i** pertenece al clúster **k**.
- **B_N**: Tamaño del bloque que procesa cada unidad de GPU

Salida

- **s** $\in \mathbb{R}^{(K \times d)}$: **s_k** suma de todos los puntos del clúster **k**
- **n** $\in \mathbb{R}^K$: **n_k** representa la cantidad de puntos en el clúster **k**
- **C (centroides)**: **c_k = s_k / n_k**
Resultado final de k-means en esta iteración

Otras Variables Importantes Usadas en el Algoritmo

- **sorted_idx**: Es un vector de índices que ordena los puntos por clúster
- **l, r**: Índices que delimitan un bloque (chunk)
 - **l**: inicio
 - **r**: fin del bloque
- **(u, v, k)**: Representa un segmento
 - **u**: inicio del segmento
 - **v**: fin del segmento
 - **k**: id del clúster

Sort-Inverse Update

1. Sort → Mapeo Inverso

Sort by cluster id: $\text{compute sorted_idx} \leftarrow \text{argsort}(a);$

Construct sorted cluster ids $a^{\text{sorted}}[j] \leftarrow a[\text{sorted_idx}[j]];$

- **Objetivo:** agrupar puntos del mismo clúster
- Se ordena el vector de asignaciones
- Se obtienen índices ordenados
- Se construye un vector de clústeres agrupados
- **Resultado después de esta etapa:** Puntos del mismo clúster quedan contiguos en memoria lógica

Antes

- Acceso aleatorio a memoria
- Escrituras desordenadas

Después

- Datos agrupados por clúster
- Acceso secuencial

$$a = [2, 1, 2, 1] \rightarrow a_{\text{sorted}} = [1, 1, 2, 2]$$

Sort-Inverse Update

2. Agregación Por Segmentos

- **Idea:** procesar datos ya organizados

```
for  $l \leftarrow 0$  to  $N - 1$  step  $B_N$  do  
     $r \leftarrow \min(l + B_N, N)$ ;  
    Load  $a^{\text{sorted}}[l : r]$  and  $\text{sorted\_idx}[l : r]$ ;  
    Identify contiguous segments of identical cluster ids in  $a^{\text{sorted}}[l : r]$ ;
```

```
for each segment  $(u, v, k)$  in  $a^{\text{sorted}}[l : r]$  do  
    Gather token features from original order using indices  $\text{sorted\_idx}[u : v]$ ;  
    Accumulate local partial sum  $\Delta s_k$  and local partial count  $\Delta n_k$  on chip;  
    Atomic merge (once per segment):  $\text{atomic\_add}(s_k, \Delta s_k)$  and  $\text{atomic\_add}(n_k, \Delta n_k)$ ;
```

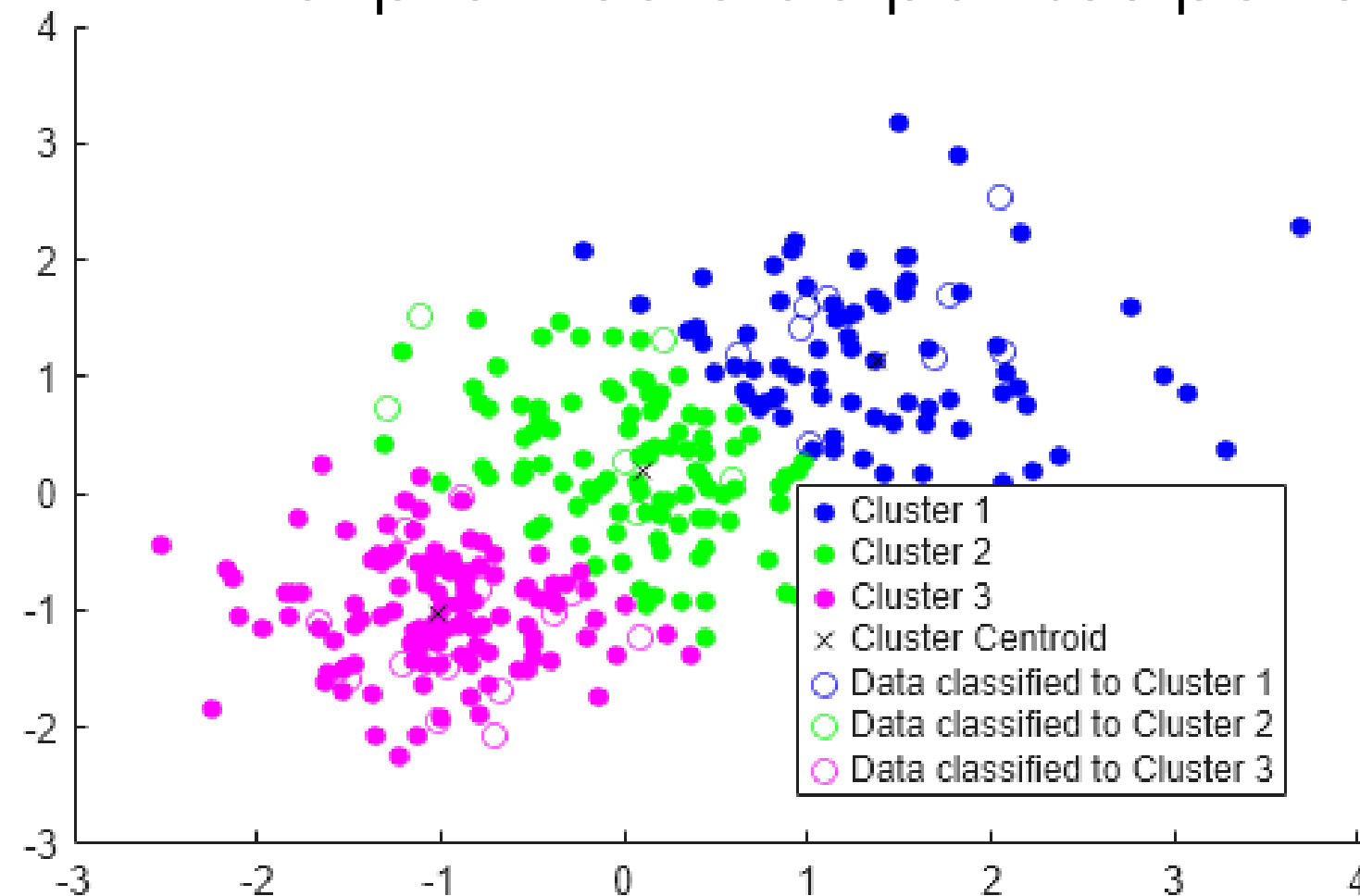
- Se procesan bloques de datos
- Dentro de cada bloque se identifican segmentos
- Cada segmento = un clúster
- Se recuperan los puntos originales
- Se suman dentro del bloque
- Se cuentan elementos
- Antes: 1 atomic por punto
- Ahora: 1 atomic por segmento
- Complejidad:
 - Antes: $O(N \cdot d)$
 - Ahora: $O(K \cdot d)$

Sort-Inverse Update

Fin del Algoritmo: Obtenemos el Promedio de Puntos Por Clúster

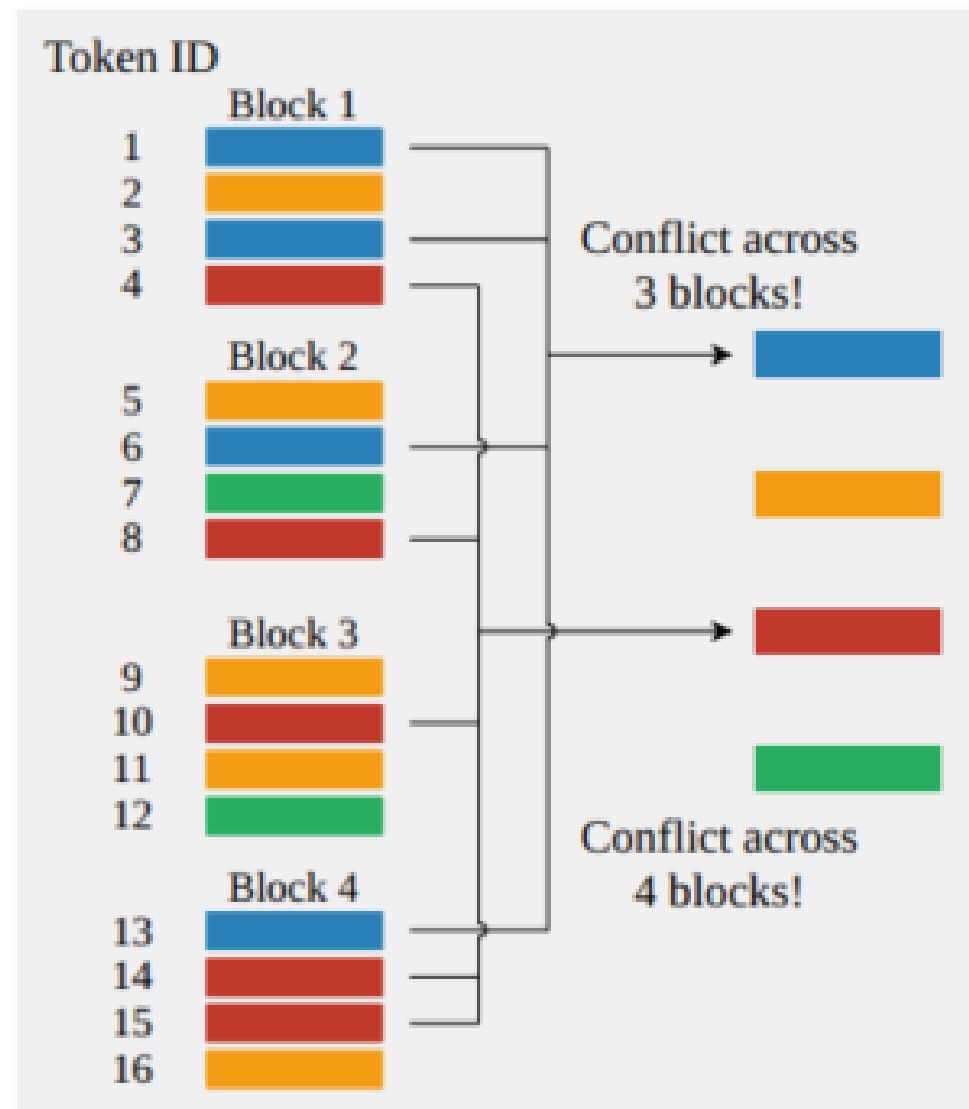
for $k \leftarrow 1$ **to** K **do**
 $c_k \leftarrow s_k / n_k;$

Es lo que termina el algoritmo y lo que cumple el objetivo de cada iteración de K-Means en sí→ Calcular el promedio de puntos por clúster.

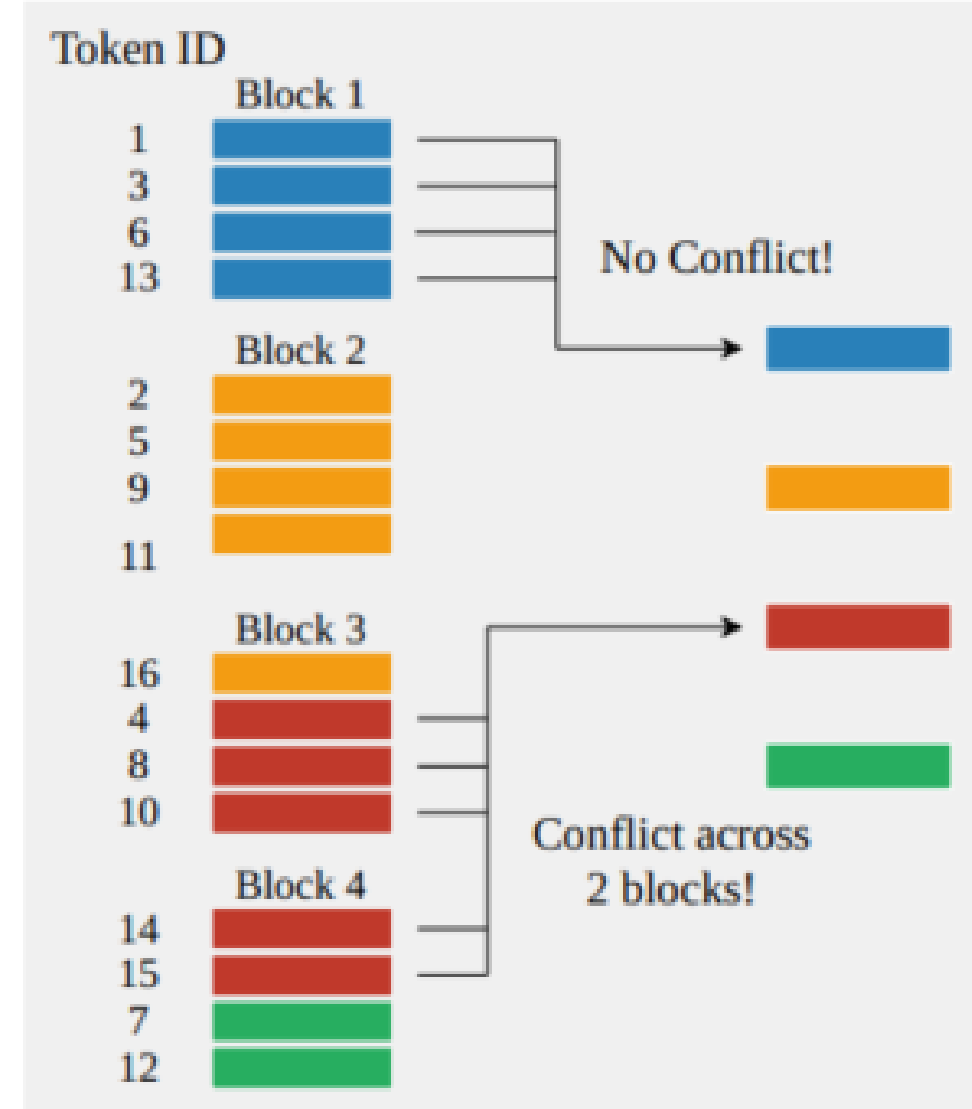


Sort-Inverse Update

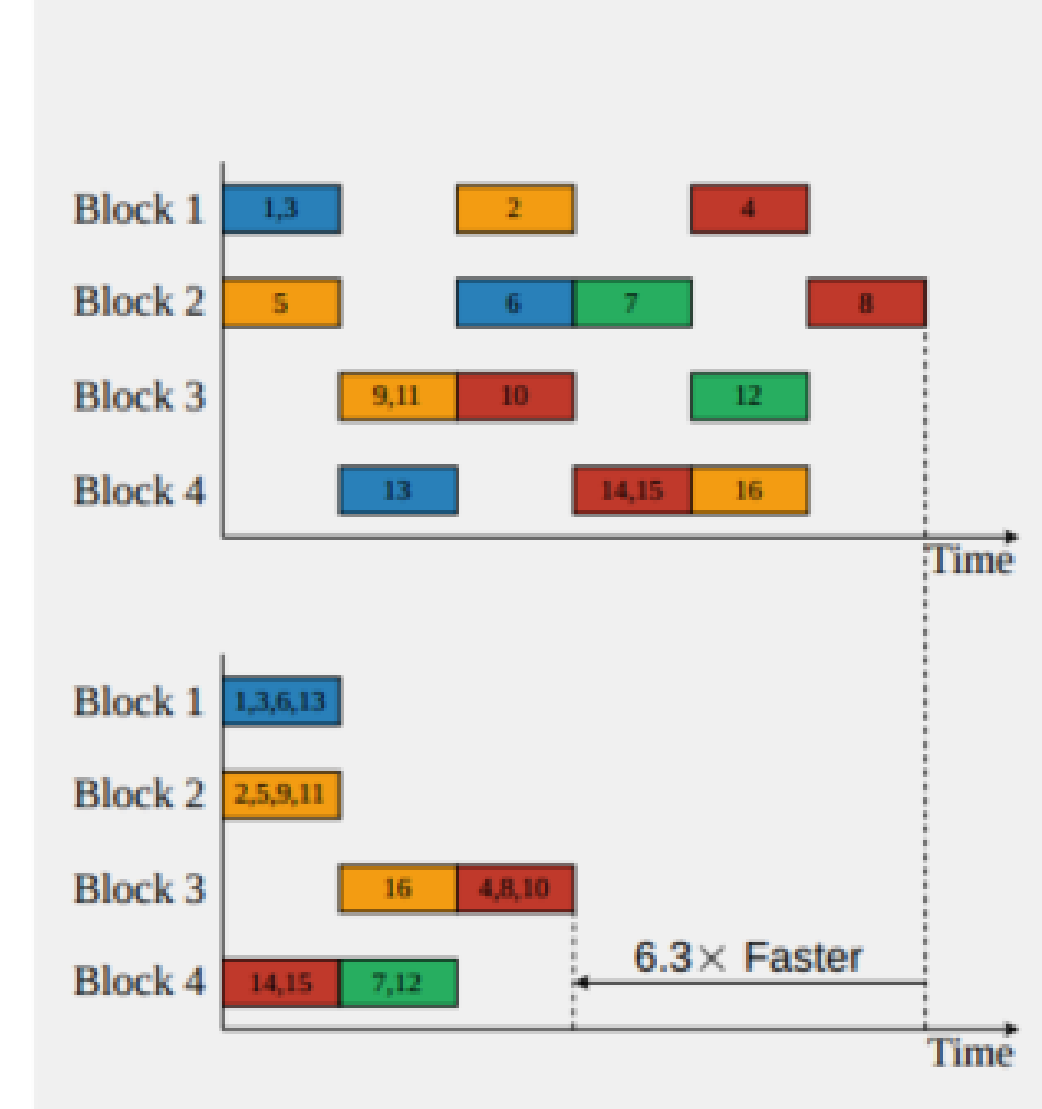
Desempeño y Comparación con Implementación de K-Means Tradicional



Actualización Estilo Scatter



Actualización con Sort-Inverse



Desempeño de Cada Algoritmo
en Tiempo de Ejecución

FLASH-KMEANS: BENCHMARK Y ESCALABILIDAD

¿Por qué K-Means falla al escalar?

K = 8192

K-Means: 30.5 GB (OOM)

Flash-KMeans: 0.48 GB

- K-Means construye una matriz de distancias enorme ($N \times K$), lo que hace que el uso de memoria crezca rápidamente.
- Flash-KMeans evita esta matriz y procesa los datos en partes, reduciendo drásticamente la memoria.

Escalando K

```
K = 1024
Estándar: 3.8 GB OK
Flash:    0.481 GB

K = 4096
Estándar: 15.3 GB OK
Flash:    0.483 GB

K = 8192
Estándar: 30.5 GB OOM (no cabe en memoria)
Flash:    0.484 GB

K = 16384
Estándar: 61.0 GB OOM (no cabe en memoria)
Flash:    0.488 GB

K = 65536
Estándar: 244.1 GB OOM (no cabe en memoria)
Flash:    0.512 GB
```

Mismo algoritmo, diferente uso del hardware.

ESCALANDO EL NÚMERO DE CLUSTERS (K)

K-Means estándar

- Construye matriz $N \times K$
- Uso de memoria crece con K
- Falla (OOM)

Flash-KMeans

- Procesa en bloques (tiles)
- No construye la matriz completa
- Funciona con K grande

Memoria

- Estándar: $O(N \times K)$
- Flash: $O(N \times D + K \times D)$

Flash-KMeans escala mejor porque elimina la dependencia directa con K en memoria

EVALUACIÓN DE RENDIMIENTO (LATENCIA)

Tiempo estándar: 907.46 ms
Tiempo Flash-like: 478.62 ms

Flash-KMeans $\approx 2\times$ más rápido

- K-Means calcula toda la matriz de distancias ($N \times K$)
- Flash-KMeans evita este cálculo completo $\rightarrow$ menos datos que mover

Menos memoria $\rightarrow$ menos transferencia $\rightarrow$ menor latencia

Flash-KMeans

MUCHAS GRACIAS

Minería de Datos